Міністерство освіти і науки України

Національний технічний університет України

«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки

Кафедра обчислювальної техніки

Лабораторна робота №5

з дисципліни «Бази даних»

Виконав:
Степаненко Денис
студент групи ІМ-051
номер у списку групи: 16

Перевірив:

Хмельницький Арсеній Андрійович

Київ 2026

Короткий виклад вимог

Завдання:

Використовуючи схему VideoHub з Lab 2, провести аналіз нормалізації: перевірити схему на відповідність нормальним формам (1NF, 2NF, 3NF), створити денормалізований приклад з навмисними порушеннями, продемонструвати аномалії та виконати нормалізацію — лікування таблиці структурою.

Чим ця лаба відрізняється від попередніх:

Лаби 1–4 щось будували: модель (Lab 1), схему (Lab 2), операції (Lab 3), аналітику (Lab 4). Тут — перевірка якості того, що збудоване. Оскільки головна схема нормалізована за дизайном, порушення створюються навмисно — щоб показати, від чого саме захищає нормальна форма.

Ключові поняття:

- **Функціональна залежність (A → B)** — значення A однозначно визначає B
- **1NF** — атомарні значення: у комірці одне значення, не список
- **2NF** — неключові атрибути залежать від усього ключа (актуально для складених PK)
- **3NF** — неключовий атрибут не залежить від іншого неключового (без транзитивних залежностей)
- **Аномалії** — наслідки дублювання: оновлення, вставки, видалення

Аналіз поточної схеми

Структура та обмеження — база описує сама себе:

```
SELECT table_name, column_name, data_type, is_nullable
FROM information_schema.columns
WHERE table_schema = 'public'
  AND table_name IN ('User', 'Video', 'Comment', 'Like', 'Subscription')
ORDER BY table_name, ordinal_position;

SELECT conrelid::regclass AS table_name, conname, contype, pg_get_constraintdef(oid)
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
ORDER BY conrelid::regclass::text, contype;
```

Перший запит повертає карту колонок і типів, другий — карту цілісності: первинні ключі (`p`), зовнішні (`f`), перевірки (`c`), унікальність (`u`). Це матеріал для аналізу: нормальні форми перевіряються по структурі, а не «на око».

Перевірка 1NF — атомарність:

```
SELECT table_name, column_name, data_type
FROM information_schema.columns
WHERE table_schema = 'public'
  AND data_type = 'ARRAY'
  AND table_name IN ('User', 'Video', 'Comment', 'Like', 'Subscription');
```

Шукаємо колонки-масиви — єдиний тип, який фізично порушує атомарність. Результат: **0 рядків**. Порожній результат тут і є доказом: у схемі немає списків у комірках — теги, телефони, ролі зберігаються окремими рядками пов'язаних таблиць.

Перевірка 2NF — часткові залежності:

2NF стосується лише таблиць зі складеним ключем: `Like(user_id, video_id)` та `Subscription(subscriber_id, channel_id)` . Єдиний неключовий атрибут цих таблиць — `created_at` — залежить від **пари**: факт лайка існує саме для пари юзер+відео, не для юзера окремо і не для відео окремо. Часткових залежностей немає — 2NF виконано.

Перевірка 3NF — транзитивні залежності:

```
SELECT column_name
FROM information_schema.columns
WHERE table_name = 'Video'
  AND column_name LIKE 'author_%';
```

Результат: **тільки** `author_id` . У Video немає `author_email` чи `author_name` — атрибути автора живуть у User, де змінюються в одному місці. Транзитивний ланцюжок `video_id → author_id → email` розірваний на межі таблиць: в межах Video його немає — 3NF виконано.

Висновок аналізу: всі 5 таблиць схеми відповідають 3NF за дизайном.

«Хвора» таблиця — денормалізований приклад

Оскільки головна схема чиста, порушення створюються навмисно — три діагнози в одній таблиці:

```
CREATE TABLE "VideoDenormalized" (
    id                      UUID PRIMARY KEY,
    title                   VARCHAR(255) NOT NULL,
    url                     VARCHAR(500) NOT NULL,
    views                   INTEGER NOT NULL DEFAULT 0,
    author_id               UUID,
    author_username         VARCHAR(100) NOT NULL,
    author_email            VARCHAR(255) NOT NULL,
    tags                    TEXT NOT NULL,
```

```
    category_and_subcategory VARCHAR(100) NOT NULL
);
```

Три діагнози:

1. `tags` **списком через кому** — порушення 1NF: у комірці список, а не одне значення
2. `category_and_subcategory` **= 'Tech/Databases'** — порушення 1NF: два факти в одній комірці
3. `author_username` , `author_email` — порушення 3NF: транзитивна залежність `id → author_id → username/email` — атрибути автора залежать не від ключа відео, а через автора

Зверніть увагу: у таблиці **немає жодних обмежень** (FK, CHECK) — це частина «хвороби»: денормалізовані копії нічого не захищають.

Наповнення даними (ті самі відео та автори, що в головній схемі — денорм-копія існуючих даних):

```
INSERT INTO "VideoDenormalized" (...) VALUES
    ('b2c3d4e5-0001-...', 'PostgreSQL Tutorial for Beginners', ..., 'denys_dev',
'denys@example.com',
     'postgresql,database,tutorial,beginners', 'Tech/Databases'),
    ... -- 5 рядків
```

Діагностика — запити-діагнози

Діагноз 1: пошук по списку — LIKE-хак:

```
SELECT title, tags FROM "VideoDenormalized"
WHERE tags LIKE '%database%';
```

Запит працює, але крихкий: підрядковий збіг знайде і `database` всередині `databases` ; індекс неможливий; унікальність тега не забезпечити; порядок тегів у списку ламає точне порівняння.

Діагноз 2: дублі довідника автора:

```
SELECT author_username, author_email, COUNT(*) AS videos
FROM "VideoDenormalized"
GROUP BY author_username, author_email;
```

Email кожного автора повторюється в кожному його відео. Зміна email = оновлення всіх рядків автора; пропустив один (збій, паралельна вставка) — у базі два email одного автора, і нічого про це не повідомить: constraint-ів немає. Це **аномалія оновлення** наживо.

Діагноз 3: два факти в комірці:

```
SELECT title, split_part(category_and_subcategory, '/', 1) AS category
FROM "VideoDenormalized"
WHERE split_part(category_and_subcategory, '/', 1) = 'Tech';
```

Запит «усі відео категорії Tech» змушений **різати рядок щоразу** — симптом того, що дані не атомарні.

Нормалізація — лікування

Крок 1: зв'язок багато-до-багатьох для тегів.

```
CREATE TABLE "Tag" (
    id   UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    name VARCHAR(50) UNIQUE NOT NULL
);

CREATE TABLE "VideoTag" (
    video_id UUID NOT NULL REFERENCES "Video"(id) ON DELETE CASCADE,
    tag_id   UUID NOT NULL REFERENCES "Tag"(id) ON DELETE CASCADE,
    PRIMARY KEY (video_id, tag_id)
);
```

`Tag` — довідник: ім'я тега унікальне і живе один раз. `VideoTag` — junction-таблиця зі складеним первинним ключем: пара (video_id, tag_id) і є ідентичність зв'язку — окремий id не потрібен. CASCADE з обох боків — сиріт не буває.

Крок 2: ALTER-хірургія на живій таблиці.

Золоте правило міграції: **спочатку ADD нового, потім перенесення даних, і лише потім DROP старого** — порушиш порядок, втратиш дані.

```
-- 2a. ADD атомарних колонок (спочатку нове)
ALTER TABLE "VideoDenormalized" ADD COLUMN category    VARCHAR(50);
ALTER TABLE "VideoDenormalized" ADD COLUMN subcategory VARCHAR(50);

-- 2b. Перенесення даних: split_part ріже 'Tech/Databases'
UPDATE "VideoDenormalized"
SET category    = split_part(category_and_subcategory, '/', 1),
    subcategory = split_part(category_and_subcategory, '/', 2);

-- 2c. DROP старого (лише після перевірки перенесення)
ALTER TABLE "VideoDenormalized" DROP COLUMN category_and_subcategory;
ALTER TABLE "VideoDenormalized" DROP COLUMN author_username;
ALTER TABLE "VideoDenormalized" DROP COLUMN author_email;
ALTER TABLE "VideoDenormalized" DROP COLUMN tags;

-- 2d. Таблиця здорова
ALTER TABLE "VideoDenormalized" RENAME TO "VideoFixed";
```

`split_part('Tech/Databases', '/', 1)` = 'Tech' — функція розрізає рядок за роздільником і бере N-ту частину. Транзитивні атрибути автора видаляються — вони живуть у User. Складена колонка категорії розрізана на дві атомарні.

Крок 3: наповнення M:N.

```
INSERT INTO "Tag" (name) VALUES
    ('postgresql'), ('database'), ('tutorial'), ('beginners'),
    ('gaming'), ('eldenring'), ('rpg'), ('bossfight'),
```

```
    ('cooking'), ('pasta'), ('italian'), ('recipe'),
    ('travel'), ('tokyo'), ('japan'), ('nightwalk'),
    ('music'), ('guitar'), ('cover')
ON CONFLICT (name) DO NOTHING;

INSERT INTO "VideoTag" (video_id, tag_id)
SELECT v.id, t.id
FROM "Video" v
CROSS JOIN "Tag" t
WHERE (v.title = 'PostgreSQL Tutorial for Beginners'
       AND t.name IN ('postgresql', 'database', 'tutorial', 'beginners'))
   OR (...)
ON CONFLICT DO NOTHING;
```

19 тегів у довідник; зв'язки — через `CROSS JOIN` з умовою: він дає всі пари відео×тег, `WHERE` залишає правильні 19 зв'язків — один запит замість ручного перебору. `ON CONFLICT DO NOTHING` робить скрипт **ідемпотентним**: повторний прогін не падає і не створює дублів.

Верифікація — дані не втрачені, переупаковані

1. Відео з тегами — чесний JOIN замість LIKE-хака:

```
SELECT v.title, t.name AS tag
FROM "Video" v
INNER JOIN "VideoTag" vt ON v.id = vt.video_id
INNER JOIN "Tag" t ON vt.tag_id = t.id
ORDER BY v.title, t.name;
```

19 рядків — той самий інформаційний контент, що був у списках через кому, тепер через індексований JOIN.

2. Атомарні категорія/підкатегорія:

```
SELECT title, category, subcategory FROM "VideoFixed"
ORDER BY category, subcategory;
```

Без розрізання рядків: 'Tech' і 'Databases' — окремі колонки.

3. Email автора — з першоджерела:

```
SELECT v.title, u.username, u.email
FROM "VideoFixed" v
JOIN "User" u ON u.id = v.author_id;
```

Email живе в одному місці — у User; JOIN повертає завжди актуальне значення.

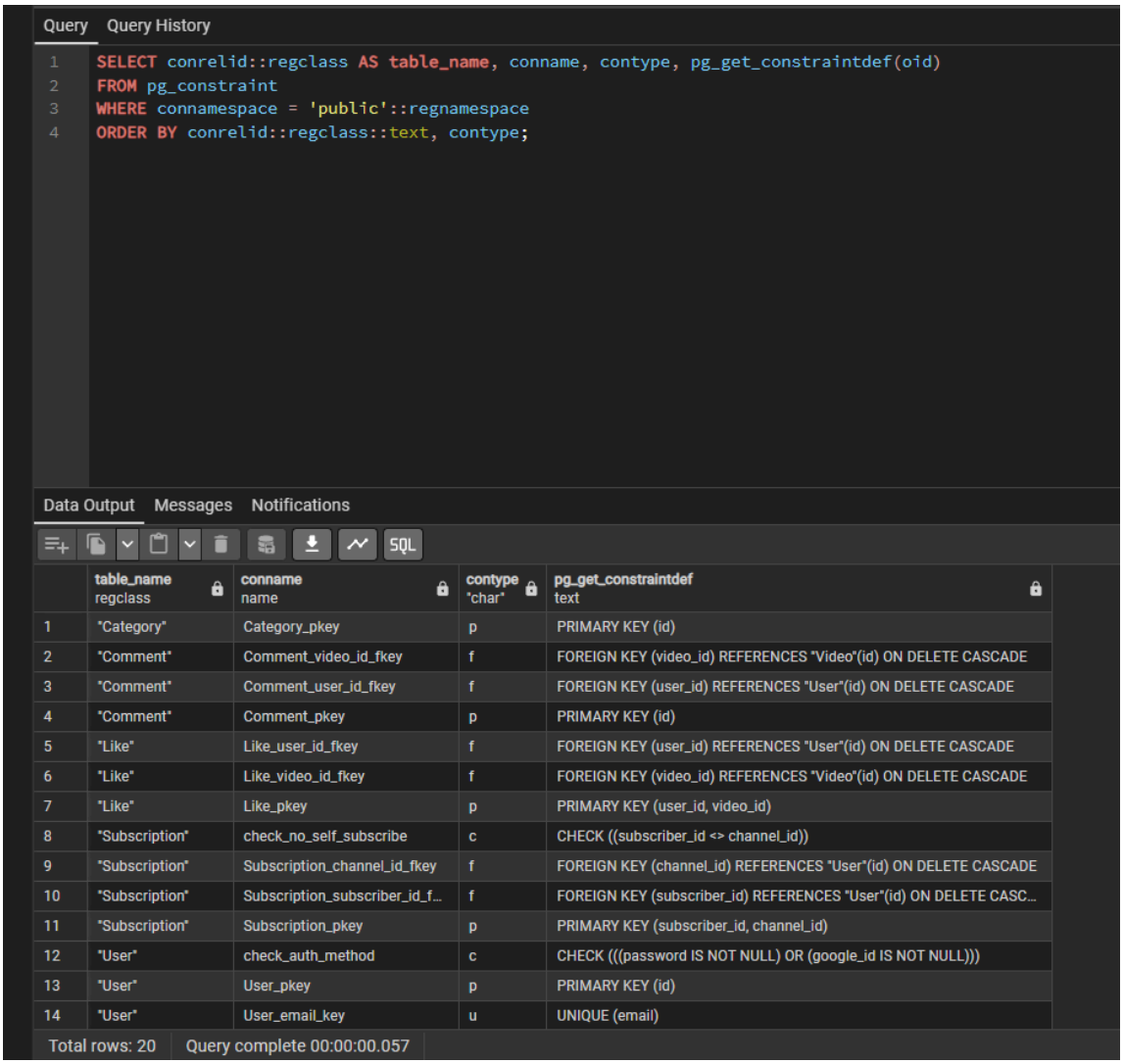
4. Топ тегів — чесний COUNT по junction:

```
SELECT t.name, COUNT(vt.video_id) AS video_count
FROM "Tag" t
LEFT JOIN "VideoTag" vt ON vt.tag_id = t.id
```

```
GROUP BY t.name
ORDER BY video_count DESC, t.name;
```

Скріншоти

Карта обмежень схеми (PK, FK, CHECK, UNIQUE всіх таблиць):



Query | Query History

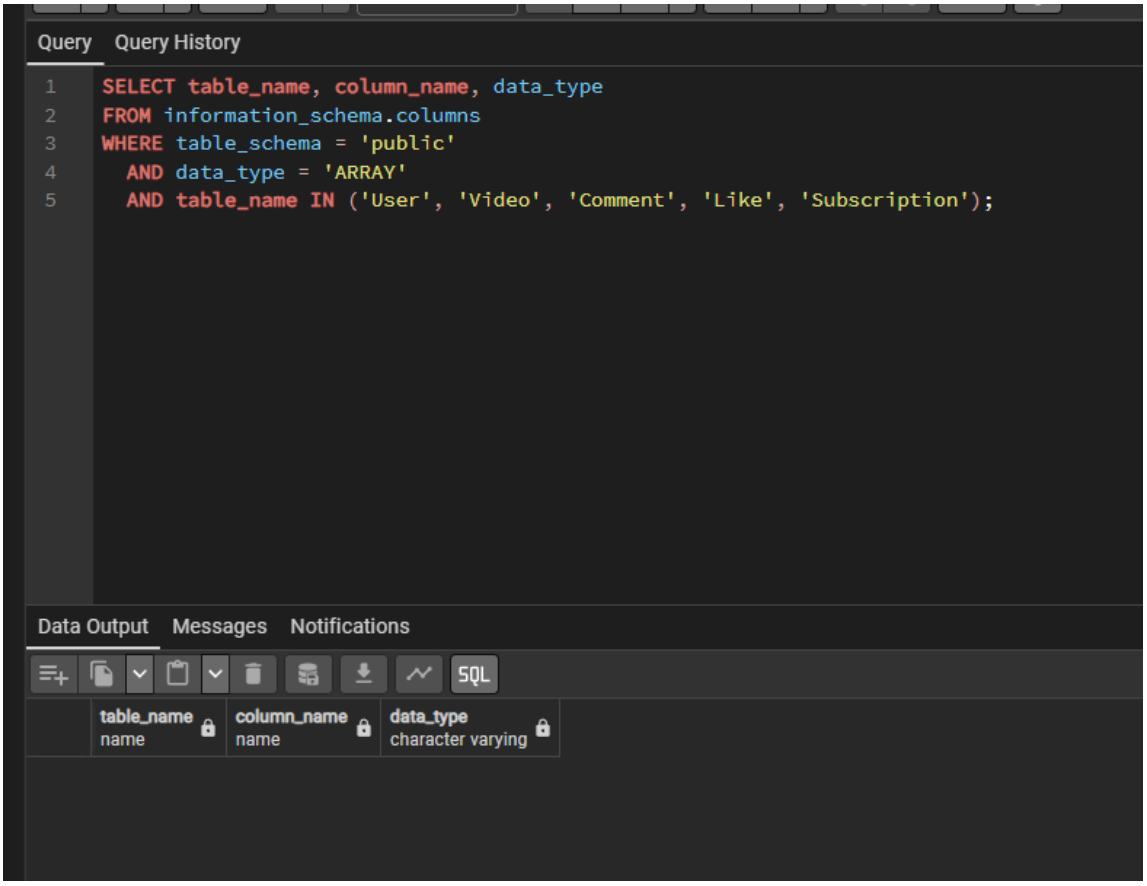
```
SELECT conrelid::regclass AS table_name, conname, contype, pg_get_constraintdef(oid)
FROM pg_constraint
WHERE connamespace = 'public'::regnamespace
ORDER BY conrelid::regclass::text, contype;
```

Data Output | Messages | Notifications

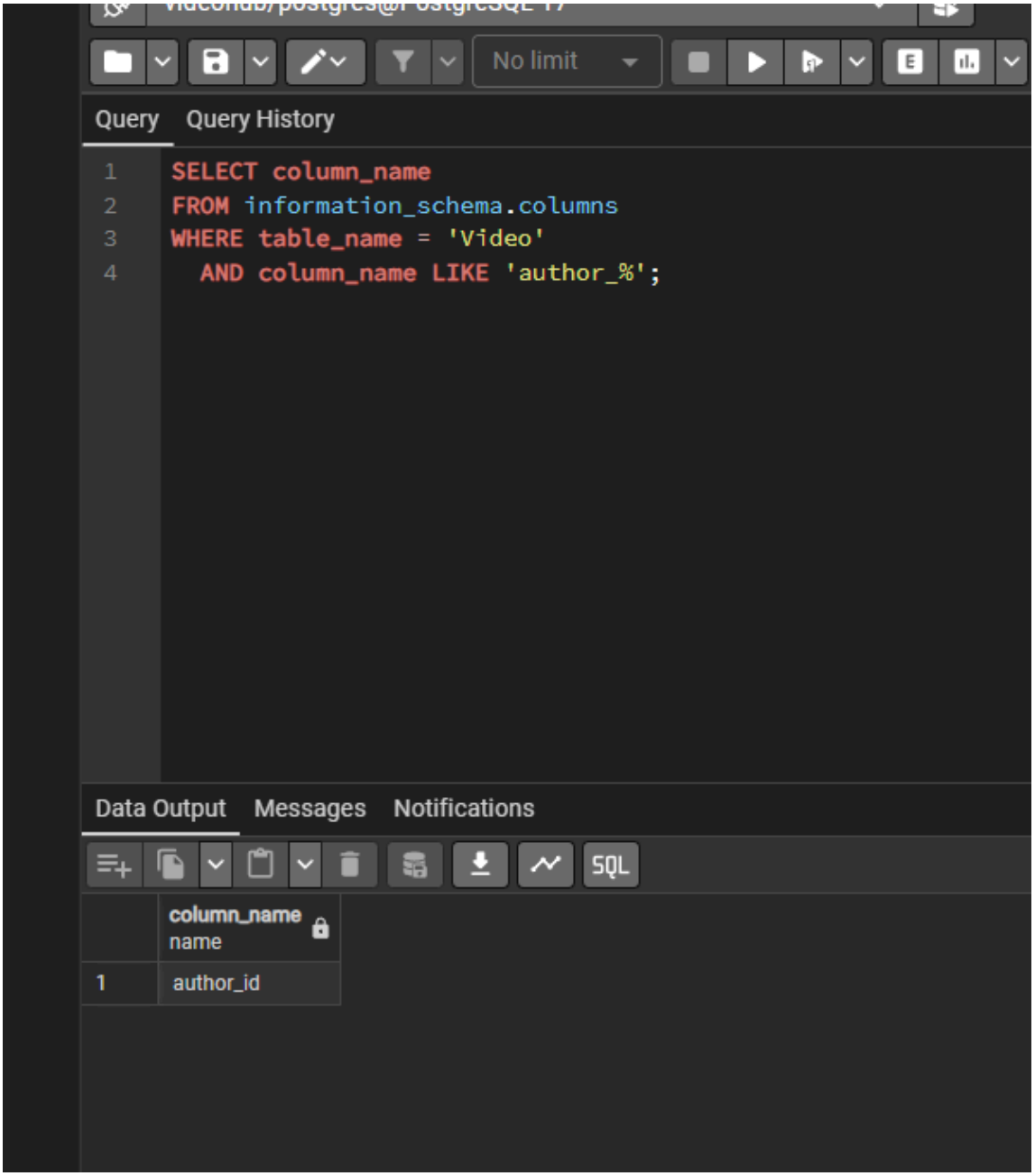
	table_name regclass	conname name	contype "char"	pg_get_constraintdef text
1	"Category"	Category_pkey	p	PRIMARY KEY (id)
2	"Comment"	Comment_video_id_fkey	f	FOREIGN KEY (video_id) REFERENCES "Video"(id) ON DELETE CASCADE
3	"Comment"	Comment_user_id_fkey	f	FOREIGN KEY (user_id) REFERENCES "User"(id) ON DELETE CASCADE
4	"Comment"	Comment_pkey	p	PRIMARY KEY (id)
5	"Like"	Like_user_id_fkey	f	FOREIGN KEY (user_id) REFERENCES "User"(id) ON DELETE CASCADE
6	"Like"	Like_video_id_fkey	f	FOREIGN KEY (video_id) REFERENCES "Video"(id) ON DELETE CASCADE
7	"Like"	Like_pkey	p	PRIMARY KEY (user_id, video_id)
8	"Subscription"	check_no_self_subscribe	c	CHECK ((subscriber_id <> channel_id))
9	"Subscription"	Subscription_channel_id_fkey	f	FOREIGN KEY (channel_id) REFERENCES "User"(id) ON DELETE CASCADE
10	"Subscription"	Subscription_subscriber_id_f...	f	FOREIGN KEY (subscriber_id) REFERENCES "User"(id) ON DELETE CASC...
11	"Subscription"	Subscription_pkey	p	PRIMARY KEY (subscriber_id, channel_id)
12	"User"	check_auth_method	c	CHECK (((password IS NOT NULL) OR (google_id IS NOT NULL)))
13	"User"	User_pkey	p	PRIMARY KEY (id)
14	"User"	User_email_key	u	UNIQUE (email)

Total rows: 20 | Query complete 00:00:00.057

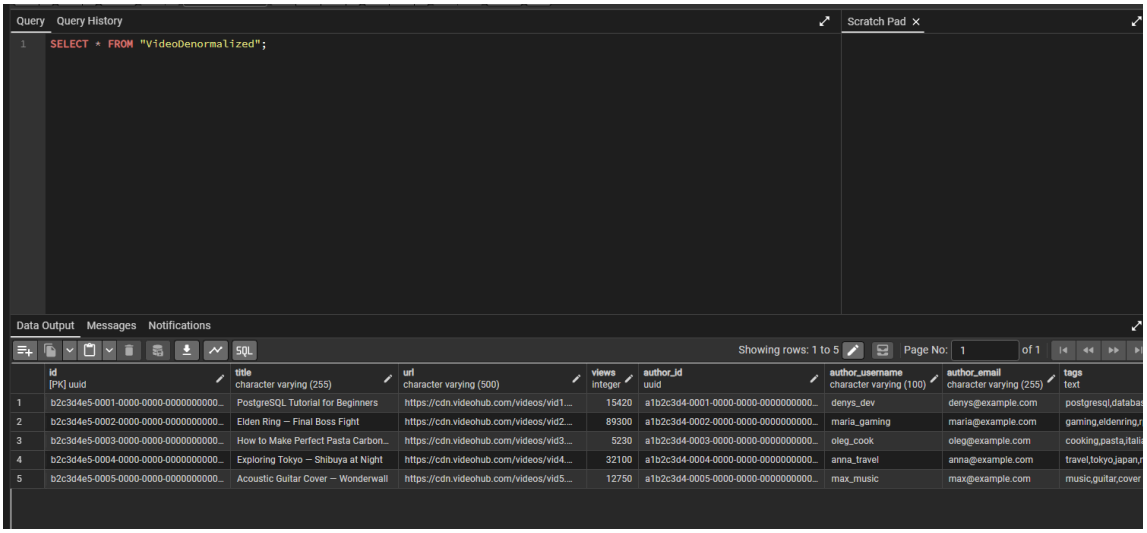
Перевірка 1NF — колонок-масивів немає (порожній результат = доказ):



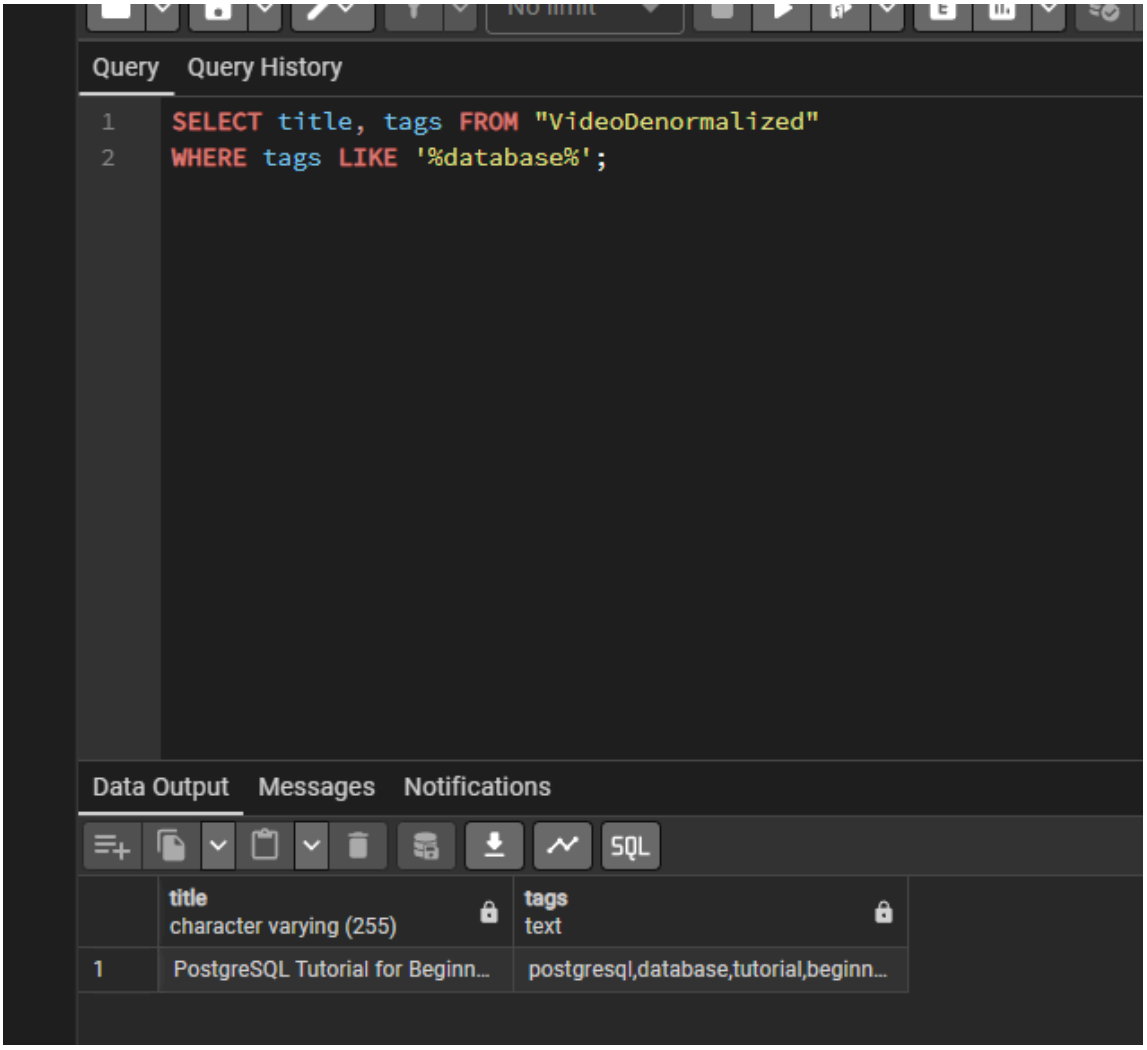
Перевірка 3NF — у Video тільки author_id:



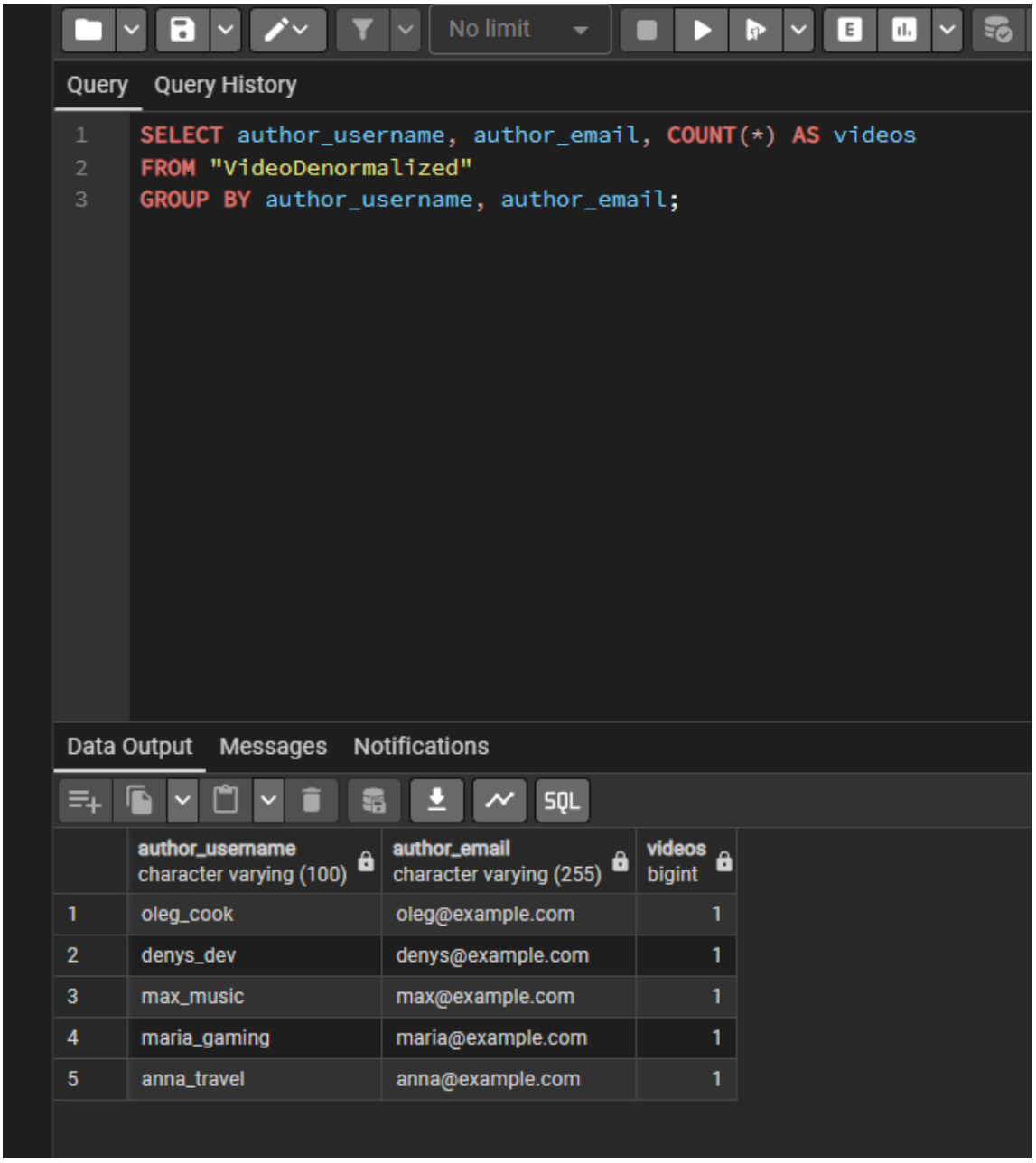
«Хвора» таблиця — три діагнози в одній таблиці:



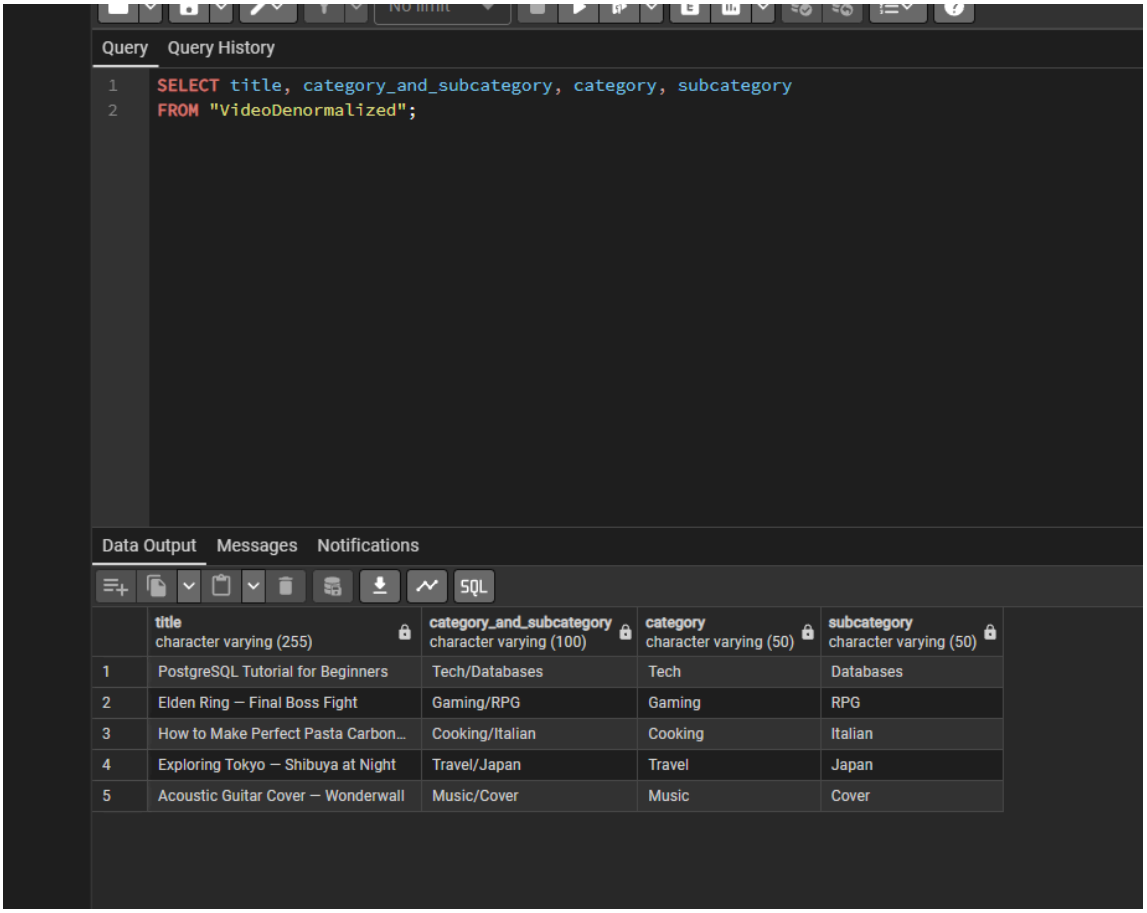
Діагностика — LIKE-хак по списку тегів:



Діагностика — дублі довідника автора (аномалія оновлення):



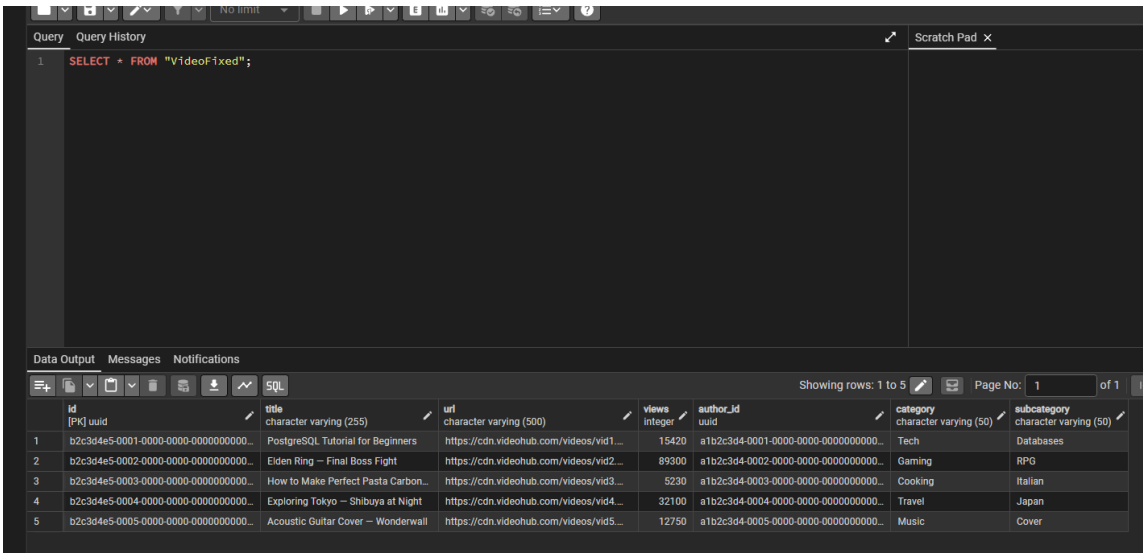
Перенесення даних — split_part розрізав категорію (дані в обох формах):



```
SELECT title, category_and_subcategory, category, subcategory
FROM "VideoDenormalized";
```

	title character varying (255)	category_and_subcategory character varying (100)	category character varying (50)	subcategory character varying (50)
1	PostgreSQL Tutorial for Beginners	Tech/Databases	Tech	Databases
2	Elden Ring — Final Boss Fight	Gaming/RPG	Gaming	RPG
3	How to Make Perfect Pasta Carbon...	Cooking/Italian	Cooking	Italian
4	Exploring Tokyo — Shibuya at Night	Travel/Japan	Travel	Japan
5	Acoustic Guitar Cover — Wonderwall	Music/Cover	Music	Cover

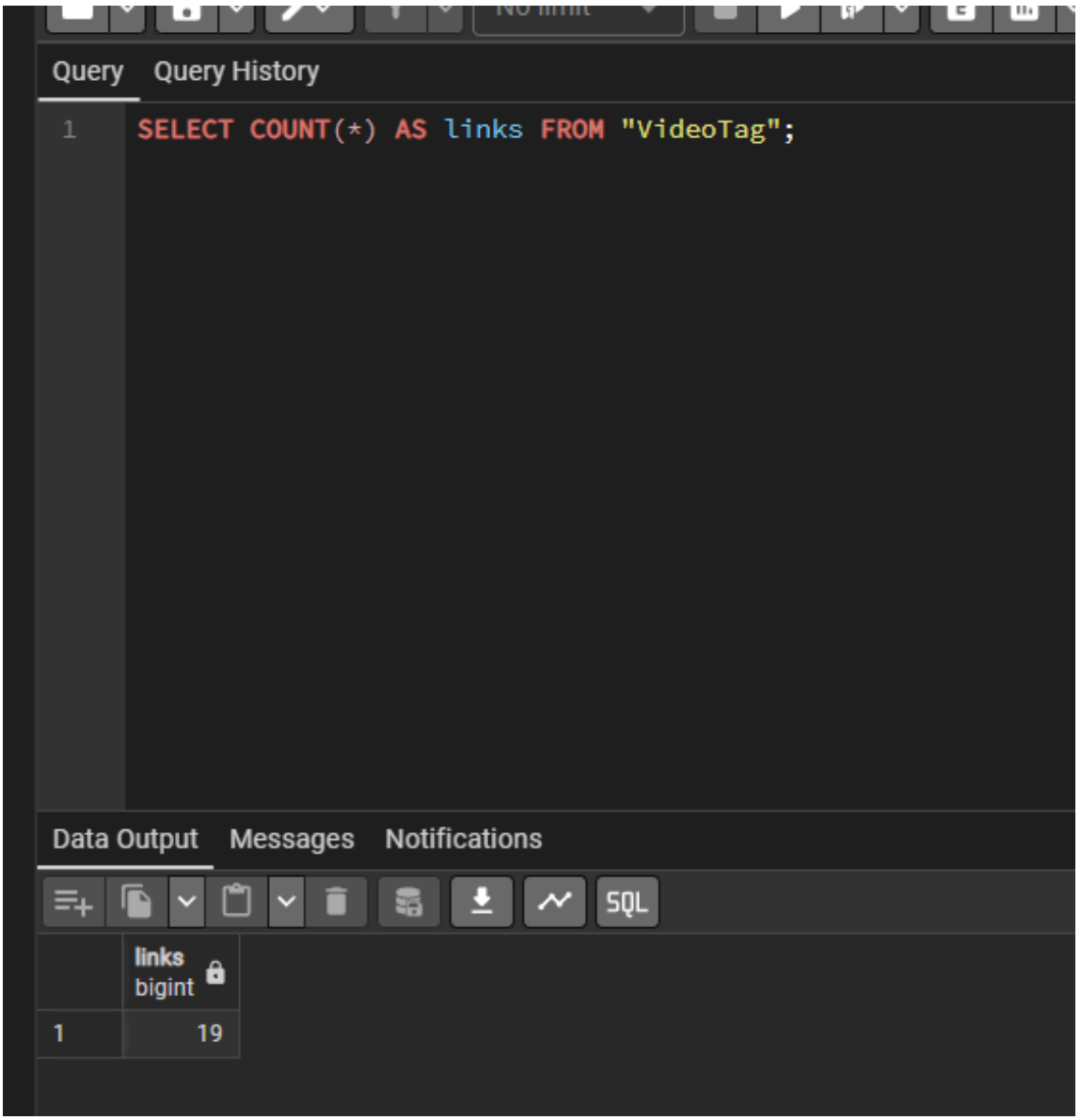
Здорова таблиця після RENAME — атомарні колонки, без транзитивних дублів:



```
SELECT * FROM "VideoFixed";
```

	id [PK] uuid	title character varying (255)	url character varying (500)	views integer	author_id uuid	category character varying (50)	subcategory character varying (50)
1	b2c3d4e5-0001-0000-0000-0000000000...	PostgreSQL Tutorial for Beginners	https://cdn.videohub.com/videos/vid1...	15420	a1b2c3d4-0001-0000-0000-0000000000...	Tech	Databases
2	b2c3d4e5-0002-0000-0000-0000000000...	Elden Ring — Final Boss Fight	https://cdn.videohub.com/videos/vid2...	89300	a1b2c3d4-0002-0000-0000-0000000000...	Gaming	RPG
3	b2c3d4e5-0003-0000-0000-0000000000...	How to Make Perfect Pasta Carbon...	https://cdn.videohub.com/videos/vid3...	5230	a1b2c3d4-0003-0000-0000-0000000000...	Cooking	Italian
4	b2c3d4e5-0004-0000-0000-0000000000...	Exploring Tokyo — Shibuya at Night	https://cdn.videohub.com/videos/vid4...	32100	a1b2c3d4-0004-0000-0000-0000000000...	Travel	Japan
5	b2c3d4e5-0005-0000-0000-0000000000...	Acoustic Guitar Cover — Wonderwall	https://cdn.videohub.com/videos/vid5...	12750	a1b2c3d4-0005-0000-0000-0000000000...	Music	Cover

Контроль наповнення M:N — рівно 19 зв'язків:



Верифікація — відео з тегами через JOIN (той самий контент):

Query | Query History

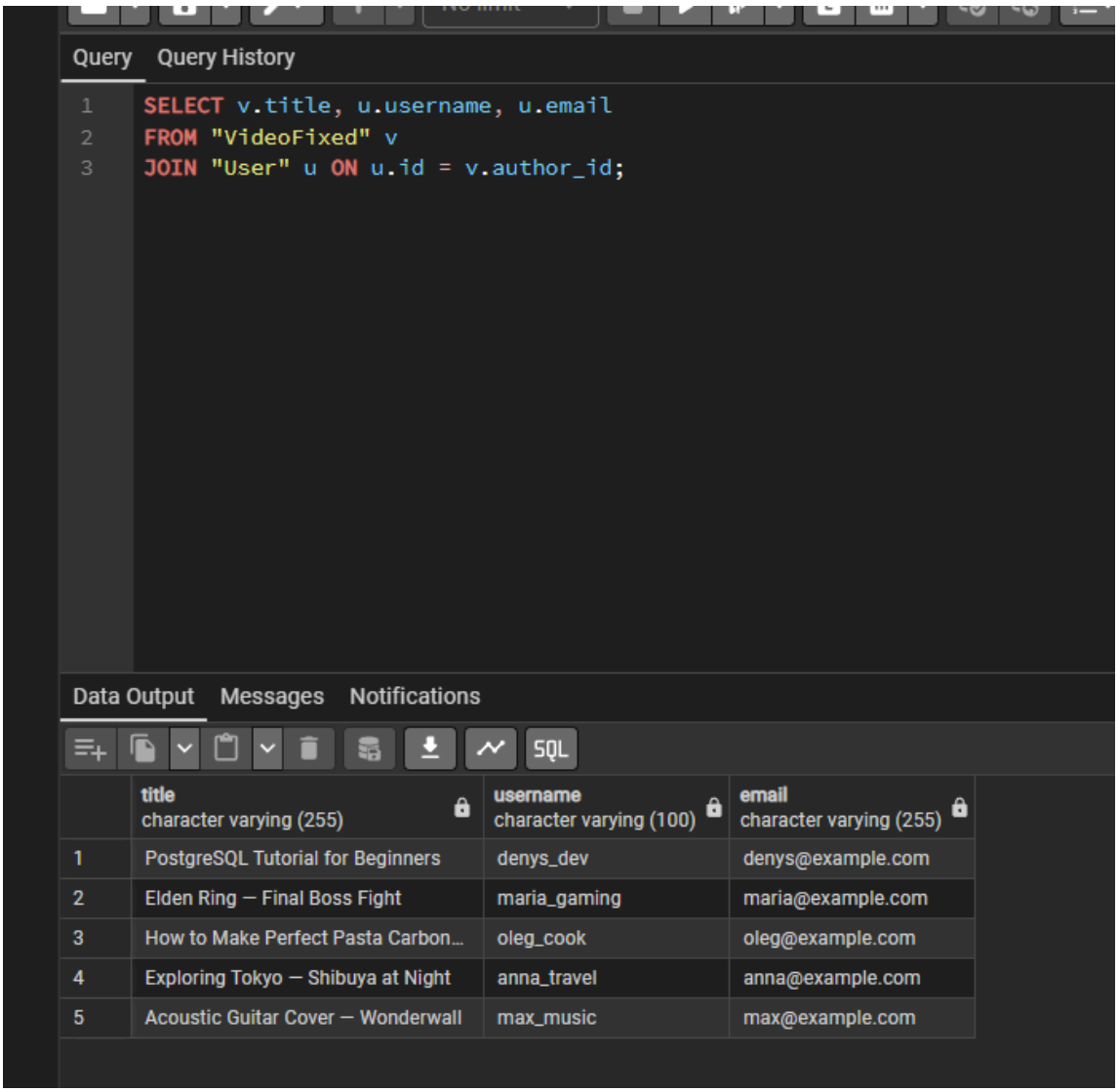
```sql
SELECT v.title, t.name AS tag
FROM "Video" v
INNER JOIN "VideoTag" vt ON v.id = vt.video_id
INNER JOIN "Tag" t ON vt.tag_id = t.id
ORDER BY v.title, t.name;
```

Data Output | Messages | Notifications

	title character varying (255)	tag character varying (50)
1	Acoustic Guitar Cover — Wonderwall	cover
2	Acoustic Guitar Cover — Wonderwall	guitar
3	Acoustic Guitar Cover — Wonderwall	music
4	Elden Ring — Final Boss Fight	bossfight
5	Elden Ring — Final Boss Fight	eldenring
6	Elden Ring — Final Boss Fight	gaming
7	Elden Ring — Final Boss Fight	rpg
8	Exploring Tokyo — Shibuya at Night	japan
9	Exploring Tokyo — Shibuya at Night	nightwalk
10	Exploring Tokyo — Shibuya at Night	tokyo
11	Exploring Tokyo — Shibuya at Night	travel
12	How to Make Perfect Pasta Carbon...	cooking
13	How to Make Perfect Pasta Carbon...	italian
14	How to Make Perfect Pasta Carbon...	pasta

Total rows: 19 | Query complete 00:00:00.036

Верифікація — email автора з першоджерела:



Будь які припущення чи обмеження

1. Порушення створені навмисно.

Головна схема VideoHub нормалізована за дизайном (перевірено запитами). Щоб продемонструвати, від чого захищають нормальні форми, створено «хвору» таблицю з трьома порушеннями: два порушення 1NF (теги списком, категорія з підкатегорією в комірці) та одне порушення 3NF (атрибути автора транзитивно через автора). Це навчальний приклад, а не частина робочої схеми.

2. «Хвора» таблиця без обмежень.

VideoDenormalized не має FK та CHECK — це частина демонстрації: денормалізовані копії нічого не захищають, цілісність тримається лише на дисципліні коду. Нормальна форма + обмеження = цілісність структурою.

3. Порядок ALTER-міграції.

Золоте правило застосовано буквально: ADD нових колонок → перенесення даних через split_part → перевірка перенесення → DROP старих → RENAME. Порушення порядку (DROP до перенесення) знищило б джерело даних — тому порядок не стилістика, а вимога безпеки.

4. Ідемпотентність наповнення.

ON CONFLICT DO NOTHING у вставці тегів і зв'язків робить скрипт повторюваним: повторний прогін не падає на унікальності і не створює дублів.

5. VideoTag посилається на головний Video.

Junction-таблиця побудована на справжніх відео головної схеми (FK → "Video"), а не на копіях — зв'язки живуть на першоджерелі. Після демонстрації таблиця VideoFixed видалена; Tag та VideoTag лишились як легітимне розширення схеми (теги тепер змодельовані правильно).